PYC

exec

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Análise Passo a Passo do Arquivo

O arquivo analisado é um script escrito em Python (`.py`) que atua como um **loader/dropper modular multi-plataforma** (compatível com Windows, macOS e Linux). Seu objetivo principal é preparar o ambiente da máquina vítima, evadir detecções geográficas, baixar dependências ocultas e executar código malicioso remoto via Node.js.

Abaixo está a análise detalhada de seus componentes principais:

1. Mecanismo de Evasão Geográfica (`_isRussianSystem`)

Verificação de Idioma e Fuso Horário: A função verifica variáveis de ambiente (`USERNAME` , `USER` , `LANG` , `LC_ALL`), fusos horários do sistema e arquivos de configuração (`/etc/timezone` , `/etc/localtime`) em busca de termos associados à Rússia (`ru_RU` , `russian` , fusos horários de Moscou, Vladivostok, etc.).

Comportamento de Evasão: Caso identifique que o sistema alvo está operando na Rússia, o script exibe a mensagem `"Russian system detected, skipping execution"` e encerra sua execução. Esta é uma técnica clássica utilizada por cibercriminosos para evitar infecções em países de origem e escapar de investigações ou sanções locais.

2. Interação com a Blockchain Solana (`_getSignFAddress` e `fvzsrnzlhr^`)

Consulta a RPCs: O script realiza requisições HTTP para diversos nós RPC públicos e oficiais da rede Solana (`mainnet-beta.solana.com` , Tatum, GetBlock, PublicNode, etc.).

Análise de Transações: Ele busca assinaturas e mensagens (`memo`) associadas a um endereço de carteira específico na blockchain, indicando possível monitoramento de transações, roubo de criptomoedas ou validação de endereços associados aos operadores do ataque.

3. Instalação Oculta de Dependências (`download_nodejs`)

Identificação de Plataforma: O script detecta automaticamente o sistema operacional (Windows, macOS, Linux) e a arquitetura do processador (`x64` , `arm64` , etc.).

Download Silencioso: Ele baixa pacotes oficiais do Node.js (`v22.9.0`) e os descompacta (via `zipfile` ou `tarfile`) em diretórios locais do usuário sem exibir janelas ou solicitar permissões visíveis, preparando um interpretador JavaScript completo para rodar os próximos estágios do ataque.

4. Comunicação com Servidor de Comando e Controle (C2) e Execução Dinâmica (`scyzzvvyr` e `main`)

Requisições e Decodificação: A função `scyzzvvyr` baixa dados de URLs externas, decodificando strings codificadas em Base64 (contendo chaves de criptografia e vetores de inicialização `_iv`).

Execução via Node.js (`i.js`): O script gera dinamicamente um arquivo JavaScript temporário (`i.js`) contendo códigos ofuscados que utilizam `eval(atob(...))` para decodificar e executar scripts arbitrários baixados em tempo de execução vindos do servidor de C2.

Determinação Final

Este arquivo É UM MALWARE.

Classificação: *Loader / Dropper* malicioso altamente sofisticado.

Justificativa: Nenhum software legítimo utiliza rotinas de verificação de geolocalização para autodestruição combinadas com o download oculto de interpretadores externos (Node.js), monitoramento de carteiras de criptomoedas na blockchain e execução dinâmica de códigos remotos desconhecidos via `eval` . Trata-se de um vetor de infecção preparado para entregar cargas maliciosas (como *infostealers* ou *trojans*) na

Qual é a categoria principal deste malware?

A categoria principal deste malware — que faz parte da campanha de ataques à cadeia de fornecimento (*supply chain*) conhecida como **GlassWorm** — é a de um **Loader / Dropper** (frequentemente designado também por *Stager*).

O Papel Principal do Malware

Este script compilado em Python atua estritamente na **fase inicial (Stage 1)** de uma intrusão complexa, desempenhando as seguintes funções fundamentais:

Evasão e Guardrails de Execução: Executa verificações geográficas e de idioma para interromper imediatamente a execução caso detete que o sistema alvo se encontra na Rússia (uma técnica comum para evitar investigações locais).

Comando e Controlo Descentralizado (C2): Efetua consultas públicas à blockchain Solana para ler informações ocultas nos campos de *memo* de transações, resolvendo dinamicamente o endereço do servidor malicioso de forma resiliente a remoções.

Preparação do Ambiente: Descarrega e instala de forma totalmente oculta e silenciosa o interpretador Node.js